

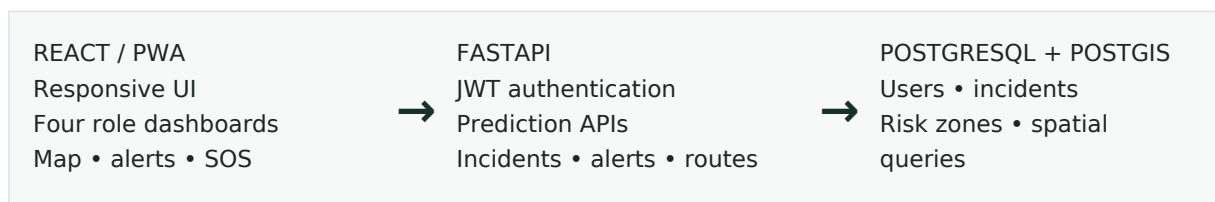
# SLIDRONIX

AI-Based Landslide Risk Assessment, Early Warning & Emergency Response System

---

## Proposed Implementation & Deployment Plan

A practical end-to-end blueprint for turning the completed environmental-data and modelling work into a clean, role-based, responsive, geospatial web platform with live inference, emergency communication, safe routing, mobile installation and public deployment.



Design direction: full-width laptop experience, responsive mobile layout, restrained Big-Tech / modern enterprise visual language, clean typography, subtle borders, professional GIS presentation, and no unnecessary “AI-themed” visual effects.

# 1. What Has Already Been Completed

The existing Slidronix repository already contains the substantial modelling and environmental-data foundation. This section is intentionally short because the new implementation is the main focus.

| Area                 | Current result                                                                                                |
|----------------------|---------------------------------------------------------------------------------------------------------------|
| Historical inventory | 34,207 historical landslide records processed from the 582-page inventory.                                    |
| Environmental data   | Terrain, MODIS NDVI/land cover, SoilGrids soil properties and CHIRPS rainfall integrated.                     |
| Modelling dataset    | 13,982 balanced spatial samples with spatially separated train/validation/test splits.                        |
| Models               | Random Forest, MLP, CNN, rainfall Transformer and multimodal Fusion model implemented and evaluated.          |
| Model comparison     | Fusion achieved the highest ROC-AUC in the completed comparison: 0.9122; MLP had slightly higher accuracy/F1. |
| GIS output           | Probabilistic risk predictions and an interactive React + Leaflet dashboard are available.                    |

Current limitation to preserve honestly: rainfall uses a fixed 1–30 January 2022 reference period, and the existing dashboard visualizes prepared predictions rather than arbitrary-coordinate live inference. The new system must address these explicitly.

## 2. Final Target System

| Citizen / User                                                                        | Researcher                                                                | Rescue Team                                                                                                    | Admin                                                                         |
|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Risk map<br>Check My Area<br>Safe Route<br>Report Incident<br>SOS<br>Emergency alerts | Datasets<br>Models<br>Risk analysis<br>Statistics<br>Research<br>Risk map | Emergency alerts<br>Incident/risk map<br>Incident status<br>Simulate scenario<br>Safe Route<br>Area monitoring | Users<br>Incidents<br>Alerts<br>Datasets/models<br>Analytics/logs<br>Settings |

Risk is an estimated probability/risk level, not a guaranteed landslide prediction or guaranteed event timing.

## 3. Repository Restructuring

Before adding the backend, reorganize the mixed research/application repository so a reviewer can immediately identify what runs the website, what trains the models, what stores the models, and what is research data.

Proposed structure

Slidronix/

- ├ frontend/ React + PWA
- ├ backend/ FastAPI application
- ├ ml/ data pipeline + training + inference
- ├ models/ production model artifacts
- ├ data/ raw + processed research data
- ├ outputs/ charts, maps, reports
- ├ docs/ architecture/documentation
- ├ scripts/ maintenance utilities
- ├ README.md
- └ .gitignore

Core modelling files to preserve: environmental extraction, dataset construction, model training, model comparison and risk-prediction generation. Debug/test/temporary scripts should be archived or removed only after checking their role.

Filename rule: names must explain function. Prefer `train_fusion_model.py`, `generate_risk_predictions.py`, `extract_chirps_rainfall.py`, `create_integrated_environmental_dataset.py` rather than unexplained labels such

as “Phase H”.

## 4. Application Architecture

| Layer             | Responsibility                                                                                | Technology                       |
|-------------------|-----------------------------------------------------------------------------------------------|----------------------------------|
| Presentation      | Dashboards, maps, forms, alerts, mobile/PWA UI.                                               | React, Leaflet, Recharts, Lucide |
| Application API   | Authentication, predictions, incidents, alerts, routing, research/admin APIs.                 | FastAPI, JWT, Pydantic           |
| ML inference      | Load model, reproduce preprocessing, create prediction inputs, return probability/risk level. | PyTorch, scikit-learn, NumPy     |
| Data/geospatial   | Users, incidents, alerts, routes, zones, team assignments, logs.                              | PostgreSQL + PostGIS             |
| External services | Rainfall/environment, geocoding, routing and push delivery.                                   | Selected APIs behind FastAPI     |

### End-to-end flow

Location / scenario → environmental feature service → tabular MLP + spatial CNN + rainfall Transformer → Fusion probability → risk level + map + action/alert

## 5. Backend and ML Foundation

| Work item            | Deliverable                                                                                                                                                  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FastAPI skeleton     | Health endpoint, configuration, CORS and clean API structure.                                                                                                |
| Model loading        | Load the Fusion model using the exact architecture used during training.                                                                                     |
| Preprocessing parity | Reproduce median imputation, tabular scaling, CNN normalization and rainfall normalization.                                                                  |
| Prediction API       | Initial test endpoint, then coordinate-based inference.                                                                                                      |
| Feature acquisition  | Generate terrain elevation/slope + 32×32 patch, satellite, soil and rainfall inputs for coordinates.                                                         |
| Real-time rainfall   | Move from the fixed reference sequence toward live/event-aligned rainfall, while validating distribution compatibility before claiming operational accuracy. |

## 6. Database, Authentication and Permissions

| Entity                     | Purpose                                                                      |
|----------------------------|------------------------------------------------------------------------------|
| users                      | Accounts, roles and authentication metadata.                                 |
| incidents                  | Reports and SOS events with location, severity, status and timestamps.       |
| alerts                     | Emergency messages, target area/audience, severity and delivery information. |
| risk_zones                 | Risk areas and geospatial boundaries for maps and route exposure.            |
| rescue_teams / assignments | Teams, operational areas and monitored users/areas.                          |
| push_subscriptions         | PWA/browser push endpoints for phone alerts.                                 |
| model_versions / datasets  | Model metadata, dataset versions and reproducibility.                        |
| logs                       | Operational and administrative audit information.                            |

JWT controls access to role-specific APIs. Authorization must be enforced in FastAPI as well as reflected in the frontend.

## 7. Citizen / User Experience

| Feature          | Implementation                                                                                                                                      |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Check My Area    | Browser location or manual coordinates → environmental features → inference → probability, risk level, contributing factors, location and guidance. |
| Risk map         | Risk zones/points, filters, legend and selected-area details.                                                                                       |
| Safe Route       | Route alternatives → intersect with risk zones → calculate exposure → recommend the lower-risk practical route.                                     |
| Report Incident  | Type, severity, description and optional image; store and trigger workflow.                                                                         |
| SOS              | Press-and-hold or guarded confirmation → GPS → critical incident → rescue/admin notification → status.                                              |
| Emergency alerts | Installed PWA receives push notification; vibration where supported and OS/browser notification behavior.                                           |

## 8. Rescue Team and Emergency Response

□ SOS: User → Geolocation → FastAPI → Incident/PostGIS → Rescue/Admin → Acknowledge → Dispatch → Resolve.

□ Emergency alert: Rescue dashboard → FastAPI → Push service → User PWA/service worker → Notification + supported vibration.

- Rescue dashboard: emergency alerts, risk/incident map, incident status, scenario simulation, safe route and area monitoring.
- Area monitoring: rescue teams can be associated with operational areas and inspect relevant incidents/risk.

Important constraint: a normal web application cannot guarantee a loud siren or bypass Android Silent/DND. Vibration and notification behavior depend on device, browser, permissions and OS settings. Guaranteed device-level alarm control would require a separate native Android scope.

## 9. Researcher and Administration

| Researcher                     | Admin                        |
|--------------------------------|------------------------------|
| Dataset summaries and metadata | User management              |
| Model versions and metrics     | Incident management          |
| Risk statistics and charts     | Alert management             |
| Risk map and spatial analysis  | Dataset/model management     |
| Research information           | Analytics, logs and settings |

## 10. UI / UX Design Rules

| Requirement     | Decision                                                                                                                        |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------|
| Desktop width   | Use the full available laptop viewport; remove the current artificial 1126px centered root container.                           |
| Responsive      | Desktop, tablet and phone layouts without horizontal overflow.                                                                  |
| Visual language | Modern Big-Tech / enterprise GIS: neutral surfaces, restrained accent, subtle borders, strong typography, minimal visual noise. |
| Risk colors     | Low green, Moderate yellow, High orange, Critical/emergency red; pair color with labels/icons.                                  |
| Maps            | Map is a primary workspace with meaningful height and available width.                                                          |
| Navigation      | Role-aware sidebar/top navigation with consistent branding/profile/logout.                                                      |
| Mobile          | Touch-friendly controls, large SOS, responsive map and installable PWA.                                                         |

## 11. Safe Routing

Safe Route is not simply a navigation link. The routing provider supplies route geometry; Slidronix evaluates that geometry against its risk information.

| Step | Action                                                               |
|------|----------------------------------------------------------------------|
| 1    | Select origin/destination or current location.                       |
| 2    | Geocode if necessary and request route alternatives.                 |
| 3    | Intersect route geometry with risk zones / risk surface.             |
| 4    | Calculate exposure score for each route.                             |
| 5    | Present shortest, lower-risk and recommended route with explanation. |

Use wording such as lower estimated landslide-risk exposure, not an absolute claim that a route is “safe”.

## 12. Public Deployment Strategy

| Service          | Purpose              | Role in Slidronix                                                            |
|------------------|----------------------|------------------------------------------------------------------------------|
| Cloudflare Pages | React frontend / PWA | Free plan and Git-connected deployment; suitable for the public UI.          |
| Render           | FastAPI backend      | Straightforward Python/FastAPI deployment; free service can sleep when idle. |
| Supabase         | PostgreSQL + PostGIS | Managed geospatial PostgreSQL; free tier has usage/pausing limits.           |
| GitHub           | Source of truth      | Version control and deployment trigger.                                      |

Target: GitHub → Cloudflare Pages (React/PWA) → Render (FastAPI) → Supabase (PostgreSQL/PostGIS), with external APIs accessed through FastAPI. Keep secret API keys on the backend.

“Permanent” means no planned 30-day trial expiry; free services still have quotas, sleep/pausing rules and policy limits. If the project becomes a real public operational service, move critical infrastructure to paid resources.

## 13. Mobile Installation Strategy

Do not start with a separate native Android application. First make the React application responsive and installable as a PWA. Add Web Push/service worker support for emergency alerts. If an APK is required for the academic demonstration, package the same web application with a web-to-native container such as Capacitor.

Sequence: responsive React → PWA manifest/service worker → Web Push → Android installation → optional APK packaging.

## 14. Testing and Demonstration Plan

| Area            | Demonstration                                                                               |
|-----------------|---------------------------------------------------------------------------------------------|
| Authentication  | Valid/invalid login and role-based access restrictions.                                     |
| Prediction      | Coordinate request returns environmental features, probability, risk level and explanation. |
| SOS             | Phone location → critical incident → rescue dashboard → acknowledgement/status update.      |
| Emergency alert | Rescue sends area alert → installed PWA receives notification and supported vibration.      |
| Incident        | User report appears on map and is manageable by rescue/admin.                               |
| Safe Route      | Two or more route alternatives are compared by estimated risk exposure.                     |
| Research        | Researcher inspects model/data statistics and risk map.                                     |
| Admin           | Admin manages users/incidents/alerts and inspects logs.                                     |
| Responsive UI   | Desktop full-screen plus phone layout tested.                                               |

## 15. Recommended Implementation Order

This order minimizes rework. Establish the data/API foundation first, then shared user capabilities, then operational roles.

### 1. Repository cleanup

Restructure files, rename unclear scripts, archive/remove temporary files, establish frontend/backend/ml/models/docs folders.

### 2. FastAPI foundation

Create entry point, configuration, health endpoint, CORS and clean API structure.

### 3. Fusion inference service

Load the exact Fusion architecture and reproduce preprocessing; verify a known sample.

### 4. Coordinate-based prediction

Implement environmental feature acquisition and the `/predict` workflow.

### 5. PostgreSQL + PostGIS

Create schema, spatial types, indexes and initial administrative data.

### 6. Authentication + roles

JWT login, role guards and protected APIs.

### 7. New full-screen React shell

Replace the centered layout with the responsive application shell while preserving the current visual language.

### 8. Citizen dashboard

Risk map, Check My Area, incident reporting and profile.

### 9. SOS workflow

GPS capture, critical incident creation, rescue notification, acknowledgement and status.

### 10. PWA + emergency push

Manifest, service worker, push subscription storage and rescue-to-user alerts.

### 11. Rescue dashboard

Incidents, risk map, area monitoring, scenario simulation, alert sending and response status.

## **12. Safe Route**

Routing API, route geometry, risk exposure scoring and recommendation.

## **13. Researcher dashboard**

Datasets, model metrics, statistics, research information and spatial analysis.

## **14. Admin dashboard**

Users, incidents, alerts, models/data, analytics, logs and settings.

## **15. Real-time environmental integration**

Connect live/event-aligned rainfall and supported sources; validate model compatibility before operational claims.

## **16. Hardening and testing**

Validation, error handling, security, mobile testing, notifications, routes and model sanity checks.

## **17. Deployment**

GitHub → Cloudflare Pages / Render / Supabase; environment variables, migrations and production URLs.

## **18. Final academic packaging**

Architecture diagrams, screenshots, test cases, results, limitations, demo script and final documentation.

## 16. Definition of Done

| Requirement        | Done when...                                                                                              |
|--------------------|-----------------------------------------------------------------------------------------------------------|
| Clean repository   | A reviewer can identify frontend, backend, ML, models, data and documentation without opening every file. |
| Live prediction    | A coordinate can produce a documented risk estimate through FastAPI.                                      |
| Four roles         | Each role sees intended features and protected APIs enforce permissions.                                  |
| Emergency response | SOS reaches rescue/admin and rescue can send targeted alerts to subscribed users.                         |
| Safe route         | Routes are compared using Slidronix risk exposure, not only distance.                                     |
| Geospatial DB      | Incidents, zones and relevant route/area geometry are queryable with PostGIS.                             |
| Responsive UI      | The interface fills laptop width and remains usable on phone screens.                                     |
| PWA                | Slidronix installs on a supported phone/browser and can receive push notifications with permission.       |
| Public deployment  | Frontend, backend and database are connected through production URLs and environment variables.           |
| Honest AI claims   | Risk is presented as an estimate and live-rainfall/model limitations are documented.                      |

## 17. Final System Vision

Slidronix should finish as one polished platform rather than a collection of demonstrations: a citizen can check local risk, report an incident or send SOS; rescue personnel can monitor areas, receive SOS, compare lower-risk routes and broadcast warnings; researchers can inspect the environmental and modelling evidence; and administrators can manage the operational system.

One platform. One geospatial database. One FastAPI backend. One shared risk engine. Four controlled roles. Responsive web + installable PWA. Publicly deployable architecture.

The completed research pipeline remains the foundation. The new work turns that foundation into a maintainable, demonstrable software system without hiding the current modelling limitations.